EXERCISE 4
TESTING AND RELIABILITY OF ELECTRONIC SYSTEMS

Testing and Reliability of Electronic Systems

ATHANASIOS FYTILIS

5381

Professor's Remarks

During the examination of our work, some issues related to exercise 4.More specifically:

- It was pointed out to me to follow the order of procedures that we are going through with **JTAG_tb.vhd.**First Sample/Preload and then BYPASS and INTEST.
- Also the clearest depiction of the situations in the testbench, because there was a confusion of signals, cycles in which nothing was happening, etc.
- And finally the deletion of EXTEST as a process that we will test in the testbench, because for the actual operation control of EXTEST we would need another circuit. Something that, according to the professor's requests, we did not need to do.

How I corrected the above issues

- Change in the order of display of the procedures in the testbench, now the IR starts from the '01' **SAMPLE/PRELOAD** process, then at the end of the '10' INTEST and '11' BYPASS respectively.(For more details see APPENDIX K3).
- I removed unnecessary circles (For more details APPENDIX K3). As a result, the code is more compact and the simulation more targeted only in terms of the questions raised by exercise 4.
- And deletion of IR _val '00' EXTEST, at the urging of the professor.

4.1 Implementation of the Circuit under Consideration (CUT)

Introduction

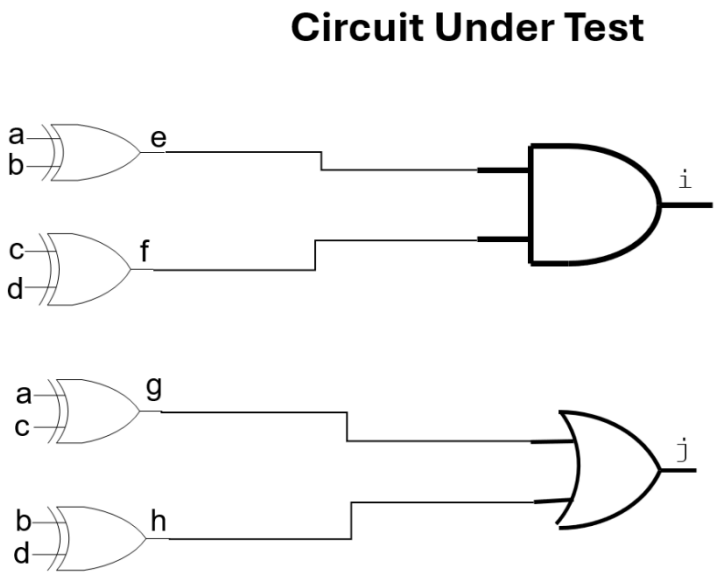
The CUT (Circuit Under Test) is the circuit we want to test through JTAG.In this exercise the CUT consists exclusively of combinatorial logic (without flip flops), as required by the assignment description.It has 4 inputs (a,b,c,d) and 2 outputs (i,j) and implements the logic that we will see below and schematically while we have fully analyzed the following circuit in the report of exercise 1.

Description of the circuit

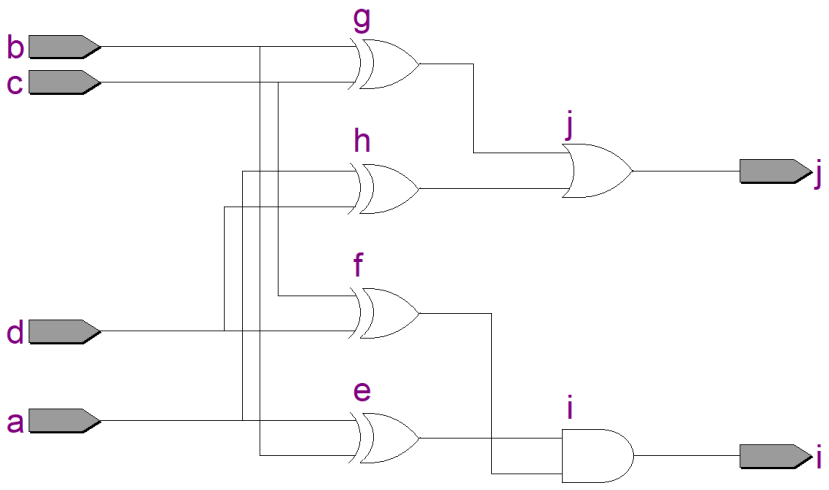
The architecture of CUT is Dataflow and consists of two layers of logic:

- 1st Level – Four XOR gates that generate the intermediate signals:
  - e = a XOR b
  - f = c XOR d
  - g = b XOR c
  - h = a XOR d
- 2nd Level – Output Gates:
  - i = e AND f → i = (a XOR b) AND (c XOR d)
  - j = g OR h → j = (b XOR c) OR (a XOR d)

Power Point:



RTL Viewer



Code(CUT.vhd) APPENDIX K1

Code Description (CUT.vhd)

CUT is implemented with Dataflow statements. There are no processes or flip-flops — all signals change instantly (combinational). The intermediate signals e, f, g, h are declared as internal signals and are computed with simultaneous assignments.

How will it be used in the context of the exercise?

As part of the overall exercise, this circuit will be surrounded by the **JTAG infrastructure**, where its inputs will be connected to the input cells of the **Boundary Scan Register (BSR)** and its outputs to the corresponding output cells.

4.2 Implementation of an integrated JTAG-CUT chip

Purpose and Functionality

The circuit was designed to allow the control of its proper functioning both at the level of internal logic and at the level of interconnections on the board (PCB).

- Normal Mode: The circuit operates transparently, performing the combined operations of the CUT without being affected by the JTAG infrastructure.
- Test Mode: The JTAG infrastructure takes control of the chip pins, allowing test vectors to be imposed and responses read

Hardware Architecture

The Integrated Circuit consists of the following main sections:

A. Circuit Under Test (CUT)

It is the central processing part, consisting exclusively of combinatorial logic (XOR, AND, OR gates) without memory elements (flip-flops). It has 4 inputs (a, b, c, d) and 2 outputs (i, j).

B. Test Access Port (TAP)

It includes the 5 terminals required to communicate with the external tester:

- **TCK (Test Clock):** The clock that synchronizes the control logic.
- **TMS (Test Mode Select):** Controls the transitions of the TAP Controller.
- **TDI (Test Data Input):** Serial input of commands and data.
- **TDO (Test Data Output):** Serial data output.
- **TRST (Test Reset):** Asynchronous initialization of the JTAG logic.

C. TAP Controller

It is a 16-state finite state machine (FSM). Generates the timing control signals (Shift, Clock, Update) for the data and command registers.

D. Registers

1. Instruction Register (IR): Stores the test commands (e.g., EXTEST, SAMPLE).
2. Boundary Scan Register (BSR): A chain of 6 scan cells (BSCs) that "embrace" the CUT, allowing it to be isolated from the terminals.
3. Bypass Register (BR): A 1-bit register that provides the shortest path from TDI to TDO.

Management of Signals and Orders

The circuit incorporates a Decoder and Muxes for the correct routing of the information:

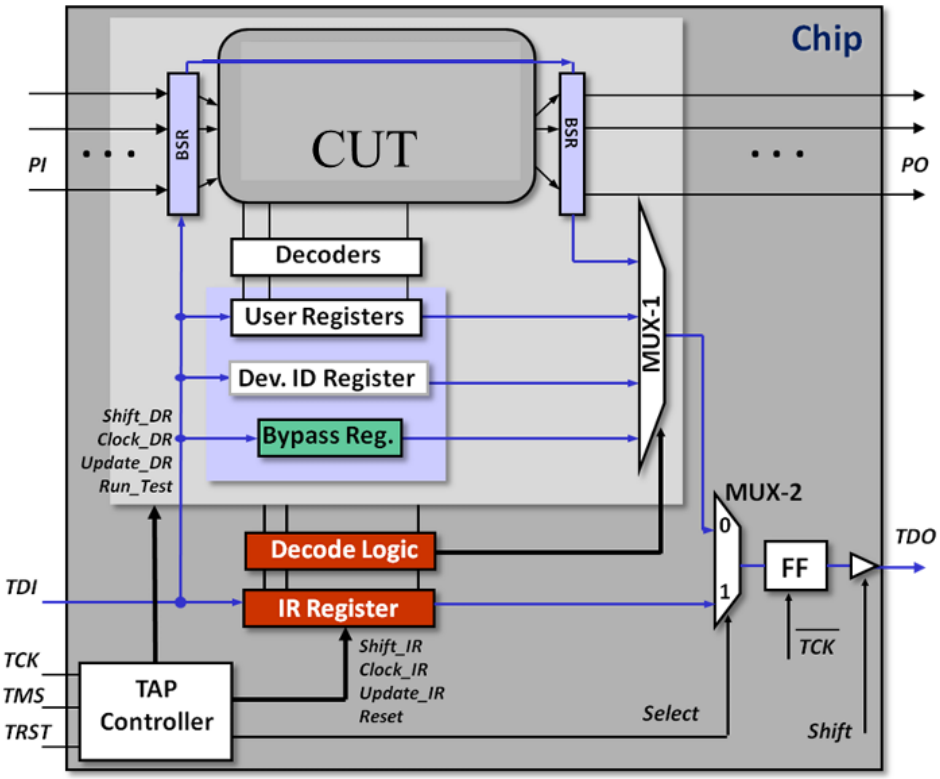
- **Decode Logic:** Translates the IR command and produces the Mode signal (which determines whether the chip is in Test or Normal mode) and the Select signal for the multiplexers.
- **Multiplexers:** Choose which register (BSR, BR, or IR) will drive the TDO output.
- **Output Synchronization:** The TDO output changes value at the negative edge of the TCK and has a tri-state buffer to remain in a High-Z state when no data shifting is required.

Supported Features

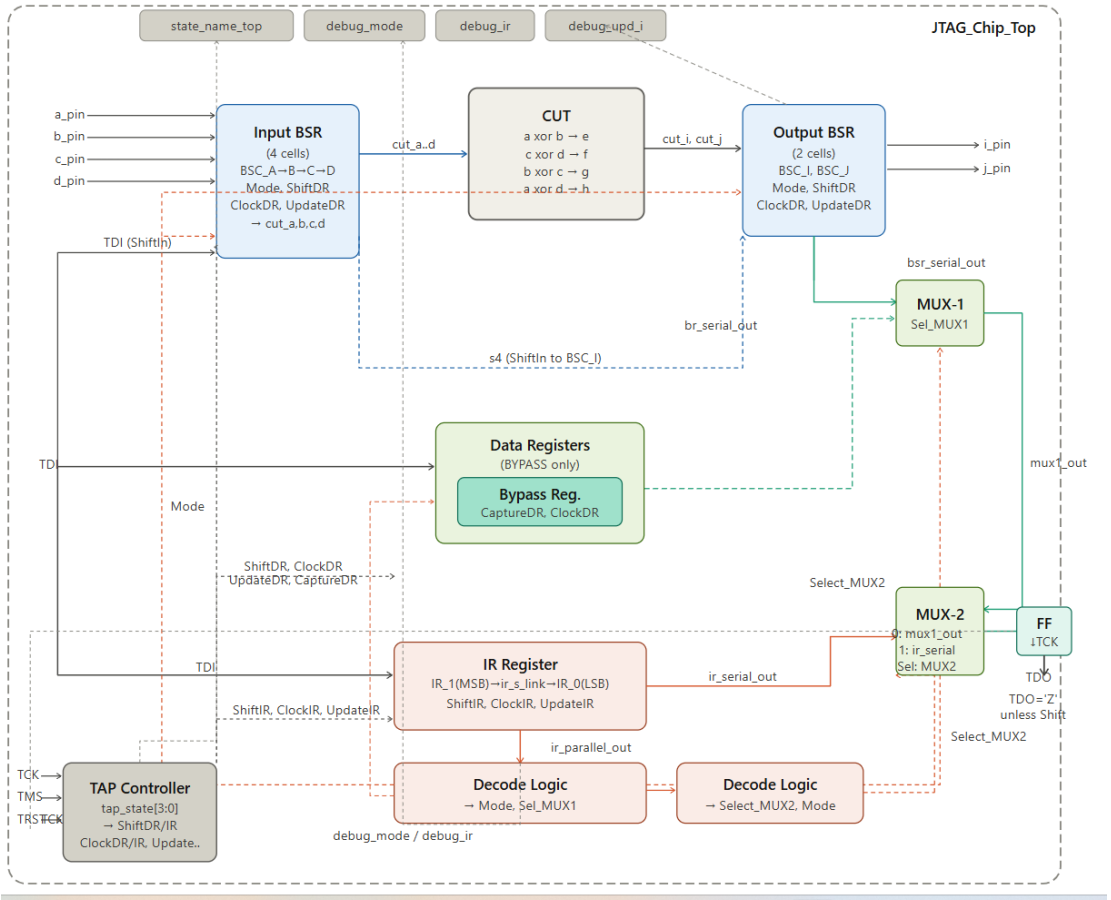
The Integrated Circuit fully implements the following procedures:

- **SAMPLE/PRELOAD:** Sampling of pin signals during normal operation.
- **EXTEST:** Control of external interfaces via BSR (Pin Permission Mode).
- **BYPASS:** Rapid access of the chip to the JTAG chain.

Power Point

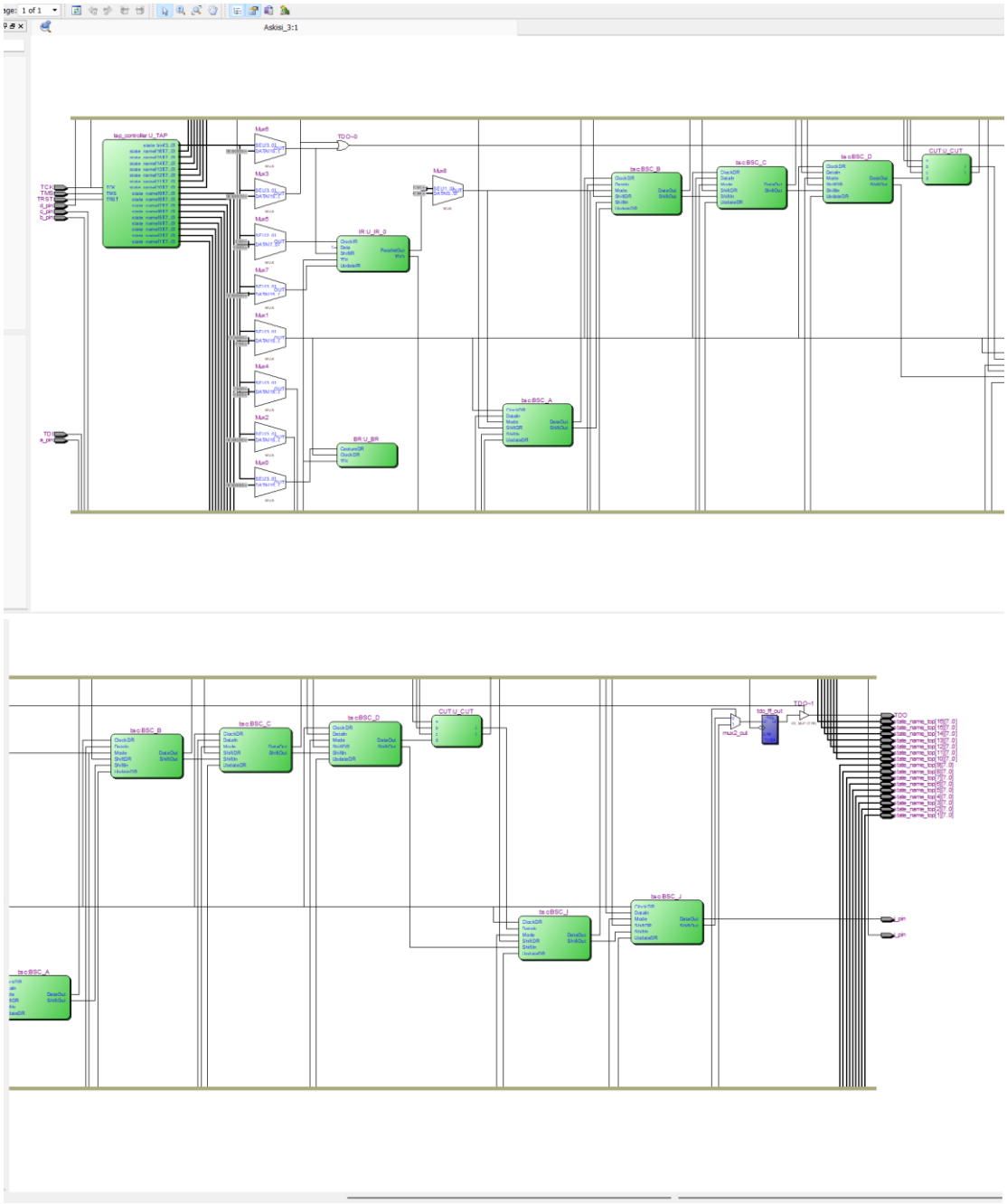

Chip
CUT
BSR
BSR
PI
PO
Decoders
User Registers
Dev. ID Register
Bypass Reg.
MUX-1
Shift_DR
Clock_DR
Update_DR
Run_Test
MUX-2
0
1
FF
TDO
Decode Logic
IR Register
TDI
TCK
TMS
TRST
TAP Controller
Shift_IR
Clock_IR
Update_IR
Reset
Select
TCK
Shift

And an effort of mine which shows somewhat more clearly the connections made in our circuit, as I implemented it.



Here we can observe somewhat more clearly that indeed our implementation in quartus is the same as that of the power point and that is, with the one elaborated in our theory.

RTL Viewer



Code(JTAG_Chip_Top.vhd) APPENDIX K2

1. Entity Ports

The circuit now includes three groups of terminals:

- **Normal I/O Pins**: The terminals a_pin up to d_pin (inputs) and i_pin, j_pin (outputs) that form the chip's interface with the external environment.
- **JTAG TAP Pins**: The mandatory terminals of the JTAG protocol: TDI, TCK, TMS, TRST, and TDO. Also included is the state_name_top output (ASCII) to monitor the status of the TAP.
- **Debug Pins (New Addition):**
    - debug_mode: Outputs the current status of the Mode signal.
    - debug_ir: Displays the current 2-bit instruction stored in IR.
    - debug_upd_i: Allows monitoring of the Update signal specific to the Output I cell.

2. Internal Architecture and Interconnection

Status Control (TAP Controller)

The U_TAP manages JTAG's FSM. An internal process generates the timing control signals (Shift, Clock, Update) for the data (DR) and command (IR) paths. In IR states, the Select_MUX2 signal is triggered to route the command to the TDO output.

Registers

- **Instruction Register (IR) - (Upgraded):** Now implemented as **a 2-bit register** consisting of two building blocks:
    - U_IR_1 (MSB): Accepts the TDI and serially connects to the next cell through the ir_s_link signal.
    - U_IR_0 (LSB): Receives the signal from the first cell and leads the ir_serial_out signal to the final multiplexer.
- **Bypass Register (BR):** Provides the 1-bit (br_serial_out) path for rapid chip bypass.

- **Boundary Scan Register (BSR):** A chain of six bsc cells. The first four (BSC_A to BSC_D) control the inputs and the last two (BSC_I, BSC_J) the outputs.

Circuit Under Test (CUT)

The U_CUT is connected between the scan cells. Its inputs are driven by the cut_a to cut_d signals, while the cut_i and cut_j outputs power the corresponding output cells of the BSR.

3. Control Logic and Routing

Decoder Logic

It analyzes the vector ir_parallel_out (2 bits) and determines the function of the chip:

Instruction	**IR Value**	**Mode**	**Sel_MUX1**	**Description**
SAMPLE	"01"	'0'	'0'	Transparent operation and signal sampling.
INTEST	**"10"**	**'1'**	**'0'**	Internal Logic Check (CUT).
BYPASS	"11"	'0'	'1'	Bypass through the Bypass Register.

Multiplexers

- **MUX-1**: Chooses between the BSR (bsr_serial_out) and Bypass Register (br_serial_out) data chain based on the Sel_MUX1 signal.
- **MUX-2**: Selects between the output of the data registers (mux1_out) and the output of the command register (ir_serial_out) based on the Select_MUX2 signal.

TDO Output

The TDO output is timed at the negative edge of the TCK via flip-flop (tdo_ff_out) to avoid race conditions. A tri-state buffer ensures that the output is only active during Shift-DR or Shift-IR states, otherwise it remains in high impedance ('Z').

Note: Initializing the IR to the ir_parallel_out signal := "01" ensures that the chip always starts in a safe SAMPLE/PRELOAD state.

Code Description(JTAG_tb.vhd)

The testbench code (JTAG_tb) is the "orchestrator" of the simulation. Its purpose is to generate the appropriate signals on the Test Access Port (TAP) to check if the chip obeys the JTAG protocol and if the internal logic (CUT) is working properly.

Here's the full breakdown of the code's parts:

1. Entity & Signals

- **Entity**: It is empty, as the testbench is a standalone unit that is not connected to external pins.
- **Signals**: Copies of all JTAG_Chip_Top inputs and outputs are created to make the connection (UUT Port Map).
- **TCK_PERIOD**: Set to **100 ns**, meaning JTAG's clock runs at 10 MHz.

2. Clock Generation and Initialization

- **TCK_process**: An endless loop that changes the value of TCK every 50 ns, creating a consistent square signal.
- **Reset**: At the beginning of the stim_proc, the **TRST** signal is set to '1' for 200 ns. This ensures that the TAP Controller will start from the **Test-Logic-Reset** state, regardless of what preceded it.

3. The Phases of the Trial (Stimulus Process)

The code is divided into specific steps that follow the logic of JTAG:

A. Load SAMPLE/PRELOAD Command ("01")

- **Objective**: To prepare the BSR without affecting the outputs.
- **Procedure**: TMS sends the sequence "0-1-1-0-0" to reach the **Shift-IR** state. There, TDI sends the bits '1' and '0'. With **Update-IR,** the command is activated and the chip is ready for sampling.

B. Uploading Data to BSR

- **Objective**: To fill the 6 BSC cells with the vector **"101100".**
- **Function**: The TAP Controller is transitions to **Shift-DR state.** The bits are pushed serially into the chain. With **Update-DR,** these values "sit" in the internal registers of the BSCs, waiting for the EXTEST command to appear on the pins.

C. BYPASS control ("11")

- **Objective**: To verify the minimum-delay bypass path.
- **Function**: The command "11" is loaded. The testbench sends a '1' to the TDI and waits to see it in the TDO after exactly one clock cycle, confirming that the path passes through the 1-bit Bypass Register.

D. INTEST Test ("10")

- **Objective**: Internal Logic Testing (CUT).
- **Procedure**:
    1. The "10" command is loaded.
    2. Values (e.g. a=1, b=1) are sent to BSR.
    3. In **Update-DR,** the values are applied to the gateway inputs.
    4. In the next **Capture-DR,** the BSCs of the outputs "cuptures" the result of the gates and send it back to the testbench through the TDO for verification.

4. Assert & Failure

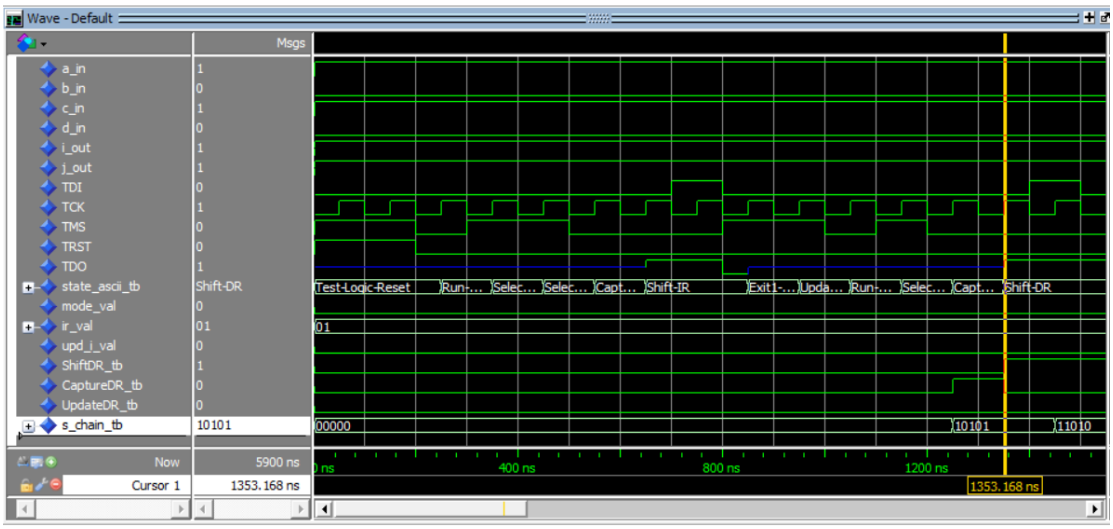
At the end, the code contains the line assert false report "Simulation Finished" severity failure?. This is not a real bug, but a "smart" way of telling the simulator (ModelSim) to stop running, otherwise the clock would keep ticking forever.

Conclusion: The testbench is complete because it checks all three mandatory commands of JTAG (EXTEST, SAMPLE, BYPASS) as well as the optional INTEST, covering the entire operating range of our design.

Code(JTAG_tb.vhd) APPENDIX K3

What do we see in the simulation?

SAMPLE/PRELOAD (01)

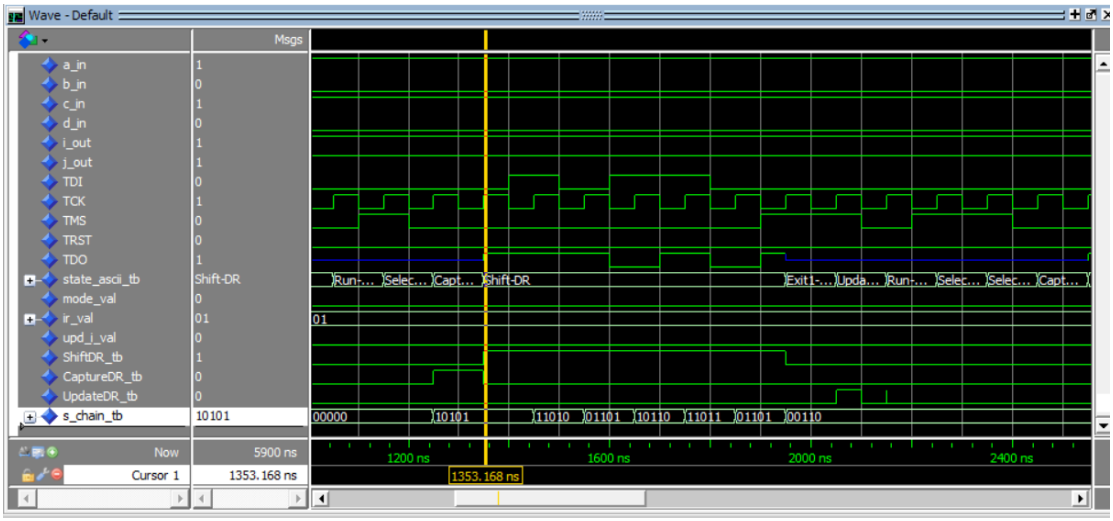


Snapshot 1 Initializing and Loading a Command

- **What we pay attention to:**
    - At start (t=0ns), the **ir_val** (Instruction Register) signal has the value **"01".** This confirms that the initialization we did is working and the system starts directly on the **SAMPLE/PRELOAD command**.
    - The **signal mode_val** (debug_mode) is fixed at **'0'**.
- **Critical point:**
    - The TAP Controller switches from the Test-Logic-Reset state to Run-Test/Idle and then to Shift-IR.

- In the interval of **Shift-IR**, we see the **TDO** signal triggering and output data, while **TDI** inserts the new command.

- **Success:**
  - The circuit is **"transparent".** Notice that the **outputs i_out** and **j_out** are already at **'1'**. This is the correct result of **the CUT** for the inputs a=1, b=0, c=1, d=0 (1 xor 0 = 1 and 1 xor 0 = 1, so 1 and 1 = 1). The JTAG logic does not interfere with normal operation.



Snapshot 2 :Running SAMPLE & PRELOAD

What we pay attention to:

- The TAP Controller is in **Shift-DR mode**. The **ir_val** remains **"01",** so the SAMPLE/PRELOAD command is active.
- The **TDI** signal flashes as we send the vector "101100" for the BSR (Preload phase).

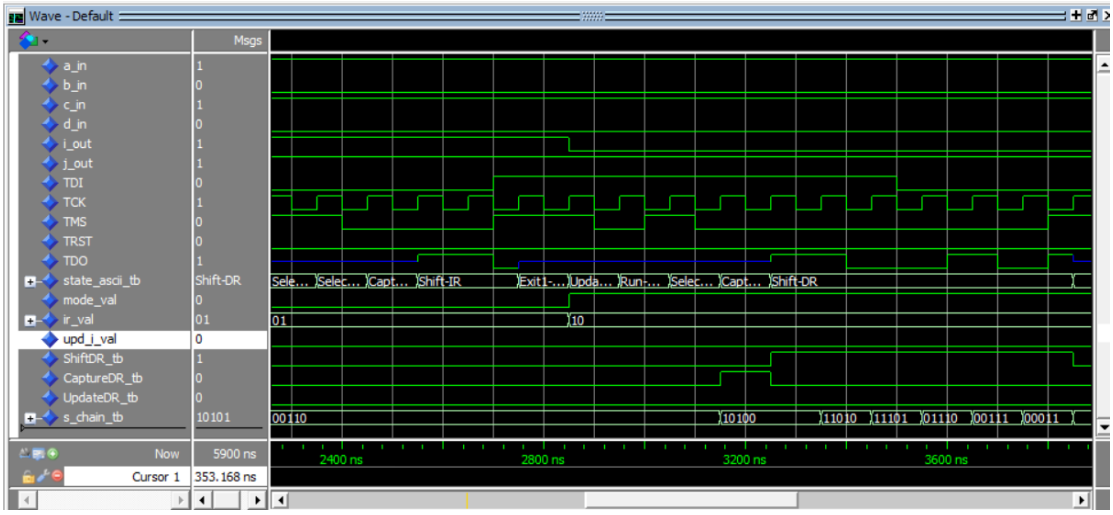
Critical point:

- Before entering Shift-DR, the controller went through **Capture-DR.** At that point, the pin values were "captured" in the register.
- Now, in **Shift-DR,** TDO outputs these captured values (**Sample** phase), allowing us to track them externally.

Success:

- Despite the data slipping within the BSR chain, the **mode_val** signal remains **'0'** and the **upd_i_val** (Update Latch) remains at **'0'**.

- The i_out **and** j_out **outputs** remain unshakable at the correct values of normal operation. BSC's output multiplexer directly connects the DataIn to the DataOut, ignoring the shifting that occurs in Shift Flip-Flops.

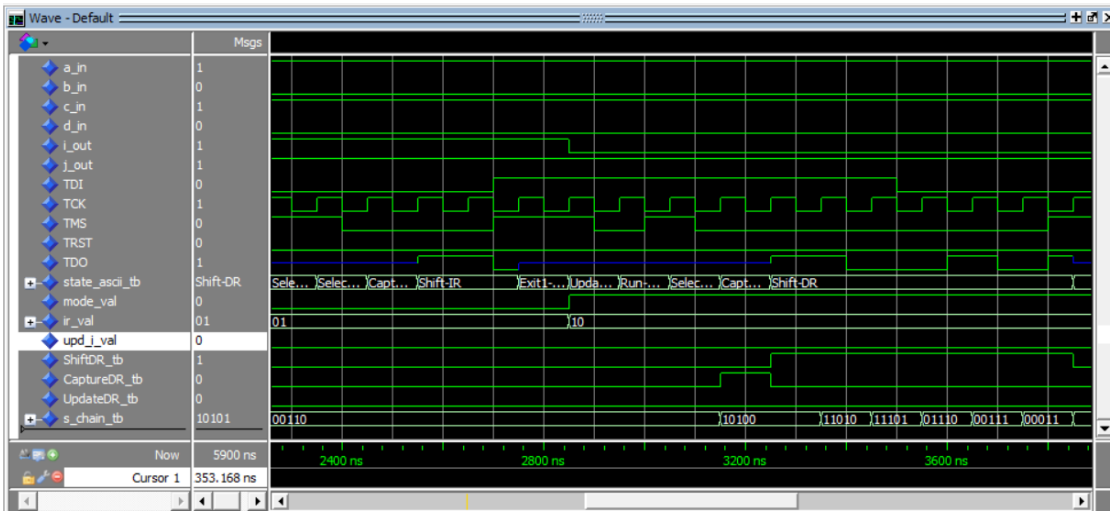


Phase 2: INTEST (Internal Test - Command "10")

- **What we pay attention to:**
  - The **ir_val** (Instruction Register) signal changes from **"01"** (Sample/Preload) to **"10"** (INTEST) immediately after the **Update-IR status**.
  - The mode_val (debug_mode) signal becomes **'1'** at the exact moment the new command is loaded.
  - **The big change:** Notice the output **i_out**. While the inputs of the pins (a_in, b_in, etc.) remain constant, the output **i_out** changes value (drops to '0') the moment the mode_val becomes '1'.
- **Critical point:**
  - This is the moment of **isolation**. The Mode multiplexer inside each BSC cell "cuts" the connection of the CUT to the outside world (Pins) and connects it to the internal Update **Latches**.

- The change in i_out output occurs because CUT now "sees" the values that we had preloaded during the previous phase. JTAG now "imposes" its own logic on the inputs of the circuit.

- **Success / What we understand:**
  - The circuit **has ceased to be transparent**. Physical inputs are completely ignored.
  - In the **Shift-DR stage** (after 3300 ns), the ShiftDR_tb signal is at **'1'**, allowing data to slide.
  - The s_chain_tb signal (the BSR chain) "comes to life": We see the values ("10100", "11010", etc.) moving from cell to cell. This is the new test vector ("110000") that you send to check the XOR gates of the CUT.
  - **What to expect:** Once the TAP Controller reaches the **Update-DR** state, a pulse will appear on the **UpdateDR_tb signal**. Then, the values that are now "running" in the chain will lock in the inputs of the CUT and we will see the next result in the outputs i_out and j_out.

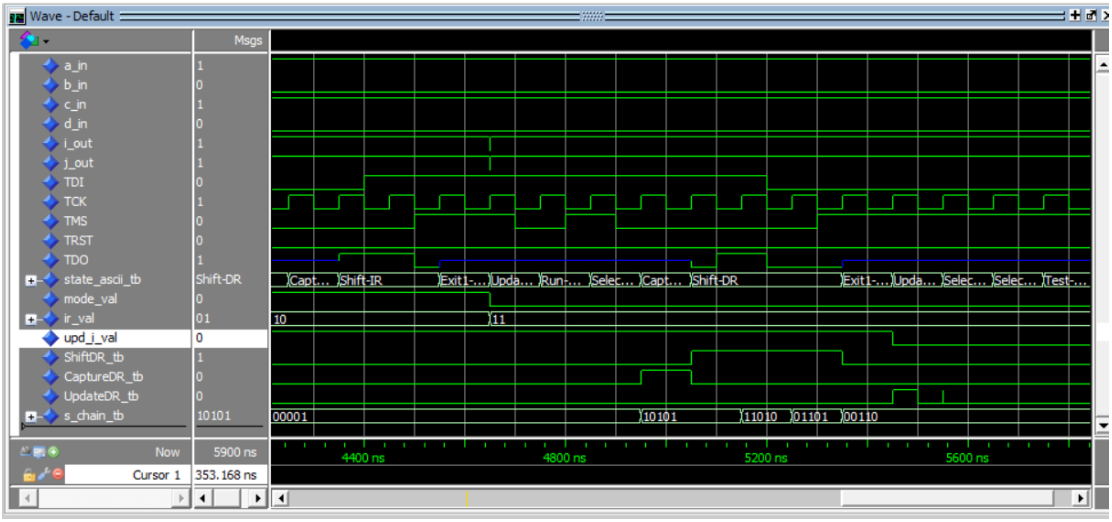


Phase 2 (End): Completion of INTEST (Command "10")

- **What we pay attention to**:
  - The pulse in the UpdateDR_tb signal (at about 2800 ns) during the **Update-DR state**.

- The change in **i_out** output, which drops to **'0'** immediately after the Update pulse.

- **Critical point**:
  - This is the moment of **execution**. Data that was "dragged" to BSR during Shift-DR is now "locked" to the CUT inputs via Update Latches.
  - The change in the i_out proves that the CUT responds to the new test vector that we imposed on it internally, even though the external inputs (a_in to d_in) remain stationary.
- **Success**: The system proved that it can fully control the logic of the chip by isolating it from the board's pins.



Phase 3 (Beginning): Switch to BYPASS (Command "11")

- **What to watch out for:** The signal ir_val to receive the value "11". With this command, the signal returns mode_val to '0', restoring the chip to Normal Mode.
- **Critical Point:** The selection of the Bypass Register (BR) through the Sel_MUX1 multiplexer. According to the design, the input of BR (D_input) is controlled by an AND gateway with the CaptureDR signal. This means that the BR samples the TDI value only during the Capture-DR state.
- **Success:** We observe a delay of exactly one TCK (1-clock delay) cycle in the waveform. The value that the TDI had during Capture is displayed in the TDO at the first falling edge during the Shift-DR state. Due to the AND gateway, any subsequent change in the TDI during Shift is ignored, confirming the correct implementation of the theory.

APPENDIX

CUT.vhd APPENDIX K1

```
library IEEE;
use IEEE. STD_LOGIC_1164.ALL;


entity CUT is
   Port (
      a : in  STD_LOGIC;
      b : in  STD_LOGIC;
      c : in  STD_LOGIC;
      d : in  STD_LOGIC;
      i : out STD_LOGIC;
      j : out STD_LOGIC
   );
end CUT;
architecture Dataflow of CUT is
   -- Internal signals for the outputs of XOR gates
   signal e, f, g, h : STD_LOGIC;
begin
   -- 1st Level: Four XOR Gates
   e <= a xor b; -- The gate leading to the e signal
   f <= c xor d; -- The gate leading to the f-signal
```

```
  g <= b xor c; -- The gate leading to the g signal
  h <= a xor d; -- The gate leading to the h signal
  -- 2nd Level: Output Gates
  i <= e and f; -- AND gate giving output i
  j <= g or  h; -- OR gate giving j output
end Dataflow;
```

JTAG_Chip_Top.vhd APPENDIX K2

```
library IEEE;
use IEEE. STD_LOGIC_1164.ALL;

entity JTAG_Chip_Top is
  Port (
    -- Normal Chip I/O Pins (Interface with the outside world)
    a_pin : in  STD_LOGIC;
    b_pin : in  STD_LOGIC;
    c_pin : in  STD_LOGIC;
    d_pin : in  STD_LOGIC;
    i_pin : out STD_LOGIC;
    j_pin : out STD_LOGIC;

    -- JTAG Pins (The Test Access Port)
    TDI   : in  STD_LOGIC;
    TCK   : in  STD_LOGIC;
    TMS   : in  STD_LOGIC;
    TRST  : in  STD_LOGIC;
    TDO   : out STD_LOGIC;
```

```
      state_name_top : out STRING(1 to 16);
      debug_mode     : out STD_LOGIC;
      debug_ir       : out STD_LOGIC_VECTOR(1 downto 0);
      debug_upd_i    : out STD_LOGIC
   );
end JTAG_Chip_Top;

architecture Structural of JTAG_Chip_Top is
   -- SIGNALS
   -- 1. Signals from the TAP Controller
   signal tap_state : STD_LOGIC_VECTOR(3 downto 0);

   -- JTAG Control Signals (Produced by tap_state)
   signal ShiftDR, ClockDR, UpdateDR, CaptureDR : STD_LOGIC;
   signal ShiftIR, ClockIR, UpdateIR : STD_LOGIC;
   signal Select_MUX2 : STD_LOGIC;

   -- 2. Signals to/from the Registers
   -- Initialize the IR to be non-red (Undefined)
   signal ir_parallel_out : STD_LOGIC_VECTOR(1 downto 0) := "01";
   signal ir_serial_out   : STD_LOGIC;
   -- 1. CHANGE: New signal for the connection of the two IR cells
         signal ir_s_link : STD_LOGIC;
   signal br_serial_out   : STD_LOGIC;
   signal bsr_serial_out  : STD_LOGIC;

   -- 3. Signals from Decoder
```

```
  -- Initialization of control signals
  signal Mode : STD_LOGIC := '0';
  signal Sel_MUX1 : STD_LOGIC := '0';

  -- 4. BSR (Boundary Scan Register) Chain Signals
  signal s1, s2, s3, s4, s5 : STD_LOGIC;

  -- 5. Signals between BSR and CUT
  signal cut_a, cut_b, cut_c, cut_d : STD_LOGIC;
  signal cut_i, cut_j : STD_LOGIC;

  -- 6. Intermediate Multiplex Signals
  signal mux1_out : STD_LOGIC;
  signal mux2_out : STD_LOGIC;
  signal tdo_ff_out: STD_LOGIC;

begin
  -- 1. TAP Controller
  U_TAP: entity work.tap_controller port map (
    TCK => TCK, TMS => TMS, TRST => TRST,
    state_bin => tap_state,
    state_name => state_name_top
  );

  -- Generate Control Signals based on TAP status
  process(tap_state, TCK)
  begin
```

```
  ShiftDR <= '0'; ClockDR <= '0'; UpdateDR <= '0'; CaptureDR <= '0';
  ShiftIR <= '0'; ClockIR <= '0'; UpdateIR <= '0'; Select_MUX2 <= '0';

  case tap_state is
    when "0011" => CaptureDR <= '1'; ClockDR <= TCK;
    when "0100" => ShiftDR   <= '1'; ClockDR <= TCK;
    when "1000" => UpdateDR  <= TCK;
    when "1010" => ClockIR   <= TCK; Select_MUX2 <= '1';
    when "1011" => ShiftIR   <= '1'; ClockIR <= TCK; Select_MUX2 <= '1';
    when "1111" => UpdateIR  <= TCK; Select_MUX2 <= '1';
    when others => null;
  end case;
end process;
            -- 2. Instruction Register (IR) - Correct Chain Connection
            -- IR_1 (MSB) starts with '0' (default)
            U_IR_1: entity work. IR
                  generic map (INIT_VAL => '0')
                  port map (
                        Data => '0', TDI => TDI, ShiftIR => ShiftIR,
                        ClockIR => ClockIR, UpdateIR => UpdateIR,
                        ParallelOut => ir_parallel_out(1), TDO => ir_s_link
                  );

            -- IR_0 (LSB) starts with '1'
            U_IR_0: entity work. IR
                  generic map (INIT_VAL => '1')
                  port map (
```

```
                                Data => '1', TDI => ir_s_link, ShiftIR => ShiftIR,
                                ClockIR => ClockIR, UpdateIR => UpdateIR,
                                ParallelOut => ir_parallel_out(0), TDO =>
ir_serial_out
                        );
   -- 3. Bypass Register (BR)
   U_BR: entity work.BR port map (
      TDI => TDI, CaptureDR => CaptureDR, ClockDR => ClockDR,
      TDO_BR => br_serial_out
   );

   -- 4. Boundary Scan Register (BSR)
   BSC_A: entity work.bsc port map (DataIn => a_pin, ShiftIn => TDI, ShiftDR =>
ShiftDR, ClockDR => ClockDR, UpdateDR => UpdateDR, Mode => Mode,
DataOut => cut_a, ShiftOut => s1);
   BSC_B: entity work.bsc port map (DataIn => b_pin, ShiftIn => s1,  ShiftDR =>
ShiftDR, ClockDR => ClockDR, UpdateDR => UpdateDR, Mode => Mode,
DataOut => cut_b, ShiftOut => s2);
   BSC_C: entity work.bsc port map (DataIn => c_pin, ShiftIn => s2,  ShiftDR =>
ShiftDR, ClockDR => ClockDR, UpdateDR => UpdateDR, Mode => Mode,
DataOut => cut_c, ShiftOut => s3);
   BSC_D: entity work.bsc port map (DataIn => d_pin, ShiftIn => s3,  ShiftDR =>
ShiftDR, ClockDR => ClockDR, UpdateDR => UpdateDR, Mode => Mode,
DataOut => cut_d, ShiftOut => s4);

   -- 5. CUT (Circuit Under Test)
   U_CUT: entity work. CUT port map (
      a => cut_a, b => cut_b, c => cut_c, d => cut_d,
      i => cut_i, j => cut_j
   );
```

```
  -- 6. BSR Continuation and Debug Upd Connection
  BSC_I: entity work.bsc port map (
    DataIn => cut_i, ShiftIn => s4, ShiftDR => ShiftDR, ClockDR => ClockDR,
    UpdateDR => UpdateDR, Mode => Mode, DataOut => i_pin, ShiftOut => s5,
    debug_upd => debug_upd_i
  );

  BSC_J: entity work.bsc port map (
    DataIn => cut_j, ShiftIn => s5, ShiftDR => ShiftDR, ClockDR => ClockDR,
    UpdateDR => UpdateDR, Mode => Mode, DataOut => j_pin, ShiftOut =>
bsr_serial_out
  );

  -- 7. Decoder Logic
  process(ir_parallel_out)
  begin
    case ir_parallel_out is
      when "00" => Mode <= '1'; Sel_MUX1 <= '0'; -- EXTEST
      when "01" => Mode <= '0'; Sel_MUX1 <= '0'; -- SAMPLE/PRELOAD
      when "10" => Mode <= '1'; Sel_MUX1 <= '0'; -- INTEST
      when "11" => Mode <= '0'; Sel_MUX1 <= '1'; -- BYPASS
      when others => Mode <= '0'; Sel_MUX1 <= '1';
    end case;
  end process;

  -- 8. MUX-1 & MUX-2 Multiplexers
```

```
  mux1_out <= br_serial_out when Sel_MUX1 = '1' else bsr_serial_out;
  mux2_out <= ir_serial_out when Select_MUX2 = '1' else mux1_out;

  -- 9. Final Flip-Flop TDO
  process(TCK)
  begin
    if falling_edge(TCK) then
      tdo_ff_out <= mux2_out;
    end if;
  end process;

  TDO <= tdo_ff_out when (ShiftDR = '1' or ShiftIR = '1') else 'Z';
  debug_mode <= Mode;
  debug_ir   <= ir_parallel_out;

end Structural;
```

JTAG_tb.vhd APPENDIX K3

```
library IEEE;
use IEEE. STD_LOGIC_1164.ALL;

entity JTAG_tb is
end JTAG_tb;

architecture Behavioral of JTAG_tb is

  -- Interface signals with the Top Module
```

```
  signal a_in, b_in, c_in, d_in : STD_LOGIC := '0';
  signal i_out, j_out         : STD_LOGIC;
  signal TDI, TCK, TMS, TRST    : STD_LOGIC := '0';
  signal TDO                  : STD_LOGIC;
        signal state_ascii_tb : STRING(1 to 16);
        signal mode_val : STD_LOGIC;
  signal ir_val   : STD_LOGIC_VECTOR(1 downto 0);
  signal upd_i_val: STD_LOGIC;
  constant TCK_PERIOD : time := 100 ns;

begin

  -- 1. Connecting the Unit Under Test (UUT)
  UUT: entity work. JTAG_Chip_Top
    port map (
      a_pin => a_in, b_pin => b_in, c_pin => c_in, d_pin => d_in,
      i_pin => i_out, j_pin => j_out,
      TDI => TDI, TCK => TCK, TMS => TMS, TRST => TRST, TDO => TDO,
                                  state_name_top => state_ascii_tb,
                                  debug_mode  => mode_val,
      debug_ir    => ir_val,
      debug_upd_i => upd_i_val
    );

  -- 2. TCK Clock Genaration
  TCK_process : process
  begin
```

```
    TCK <= '0'; wait for TCK_PERIOD/2;
    TCK <= '1'; wait for TCK_PERIOD/2;
  end process;

  -- 3. Main Test Procedure (Stimulus)
 stim_proc: process
  begin
    -- INITIALIZE PINS (Normal Mode)
    a_in <= '1'; b_in <= '0'; c_in <= '1'; d_in <= '0';
    TMS <= '1'; TDI <= '0'; TRST <= '1';
    wait for TCK_PERIOD * 2;
    TRST <= '0'; -- Disable Reset

    -- ============================================================
    -- STEP 1: SAMPLE/PRELOAD (COMMAND "01")
    -- ============================================================
    -- Switch to Shift-IR
    TMS <= '0'; wait for TCK_PERIOD; -- Idle
    TMS <= '1'; wait for TCK_PERIOD; -- Select-DR-Scan
    TMS <= '1'; wait for TCK_PERIOD; -- Select-IR-Scan
    TMS <= '0'; wait for TCK_PERIOD; -- Capture-IR
    TMS <= '0'; wait for TCK_PERIOD; -- Shift-IR

    -- Load command "01" (LSB: 1, MSB: 0)
    TDI <= '1'; wait for TCK_PERIOD; -- Bit 0
    TDI <= '0'; TMS <= '1'; wait for TCK_PERIOD; -- Bit 1 & Exit1-IR
    TMS <= '1'; wait for TCK_PERIOD; -- Update-IR
```

```
TMS <= '0'; wait for TCK_PERIOD; -- Back to Idle


-- Sample and Preload
TMS <= '1'; wait for TCK_PERIOD; -- Select-DR-Scan
TMS <= '0'; wait for TCK_PERIOD; -- Capture-DR (This is where the SAMPLE is made)
TMS <= '0'; wait for TCK_PERIOD; -- Shift-DR (This is where the PRELOAD happens)


-- We send vector "101100" (for the 6 BSCs)
TDI <= '1'; wait for TCK_PERIOD; -- Bit 1
TDI <= '0'; wait for TCK_PERIOD; -- Bit 2
TDI <= '1'; wait for TCK_PERIOD; -- Bit 3
TDI <= '1'; wait for TCK_PERIOD; -- Bit 4
TDI <= '0'; wait for TCK_PERIOD; -- Bit 5
TDI <= '0'; TMS <= '1'; wait for TCK_PERIOD; -- Bit 6 & Exit1-DR


TMS <= '1'; wait for TCK_PERIOD; -- Update-DR
TMS <= '0'; wait for TCK_PERIOD; -- Idle


-- ========================================================
-- STEP 2: INTEST (COMMAND "10")
-- ========================================================
-- Load command "10" on IR
TMS <= '1'; wait for TCK_PERIOD; -- Select-DR-Scan
TMS <= '1'; wait for TCK_PERIOD; -- Select-IR-Scan
TMS <= '0'; wait for TCK_PERIOD; -- Capture-IR
TMS <= '0'; wait for TCK_PERIOD; -- Shift-IR
```

```
TDI <= '0'; wait for TCK_PERIOD; -- Bit 0
TDI <= '1'; TMS <= '1'; wait for TCK_PERIOD; -- Bit 1 & Exit1-IR
TMS <= '1'; wait for TCK_PERIOD; -- Update-IR

-- Apply values to CUT (a=1, b=1, c=0, d=0)
TMS <= '0'; wait for TCK_PERIOD; -- Idle
TMS <= '1'; wait for TCK_PERIOD; -- Select-DR-Scan
TMS <= '0'; wait for TCK_PERIOD; -- Capture-DR
TMS <= '0'; wait for TCK_PERIOD; -- Shift-DR
TDI <= '1'; wait for TCK_PERIOD; -- a='1'
TDI <= '1'; wait for TCK_PERIOD; -- b='1'
TDI <= '0'; wait for TCK_PERIOD; -- c='0'
TDI <= '0'; wait for TCK_PERIOD; -- d='0'
TDI <= '0'; wait for TCK_PERIOD; -- Dummy i
TDI <= '0'; TMS <= '1'; wait for TCK_PERIOD; -- Dummy j & Exit1-DR
TMS <= '1'; wait for TCK_PERIOD; -- Update-DR

-- ============================================================
-- STEP 3: BYPASS (COMMAND "11")
-- ============================================================
-- Load command "11" into IR
TMS <= '1'; wait for TCK_PERIOD; -- Select-DR-Scan
TMS <= '1'; wait for TCK_PERIOD; -- Select-IR-Scan
TMS <= '0'; wait for TCK_PERIOD; -- Capture-IR
TMS <= '0'; wait for TCK_PERIOD; -- Shift-IR
TDI <= '1'; wait for TCK_PERIOD; -- Bit 0
TDI <= '1'; TMS <= '1'; wait for TCK_PERIOD; -- Bit 1 & Exit1-IR
```

```
        TMS <= '1'; wait for TCK_PERIOD; -- Update-IR


        -- Bypass Path Verification (TDI -> TDO)
        TMS <= '0'; wait for TCK_PERIOD; -- Idle
        TMS <= '1'; wait for TCK_PERIOD; -- Select-DR-Scan
        TMS <= '0'; wait for TCK_PERIOD; -- Capture-DR
        TMS <= '0'; wait for TCK_PERIOD; -- Shift-DR
        TDI <= '1'; wait for TCK_PERIOD; -- This '1' should be seen in the TDO after 1
cycle
        TDI <= '0'; wait for TCK_PERIOD; --
        TMS <= '1'; wait for TCK_PERIOD; -- Exit1-DR


        wait for 500 ns;
        assert false report "Simulation Finished" severity failure;
    end process;


end Behavioral;
```